CSE 242 Assignment 3 Writeup

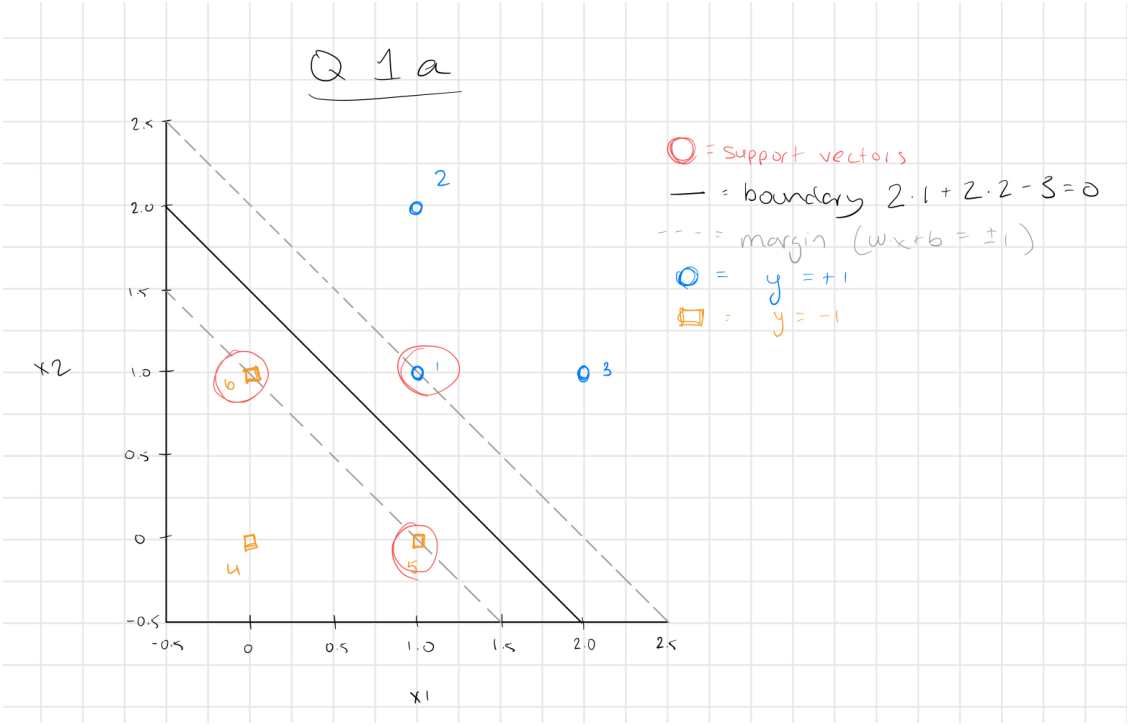
Sanya Murdeshwar

I acknowledge the use of gen AI for this assignment and have attached my transcript.

Question 1: Hard-margin linear SVM — 30 points

(a) Plot and support vectors — 10 points.

Plot the data by hand or with Python and identify all support vectors by eye. Explain your selection.



I selected these support vectors to be 1, 5, and 6 by eye because point 1 is the positive-class point that's closest to the negative group, while 5 and 6 are the closest negative-class points to the positive group. The others don't matter because they're far from the other class, and removing wouldn't change anything because they're not supports.

(b) Boundary and margin — 20 points.

Using the support vectors, derive the maximum-margin separating line w1 x1 + w2 x2 + b = 0 (10 points), then calculate the geometric margin (10 points). Include the intercept b. Assume

a linear, hard-margin SVM with no slack variables. Report the margin as the distance from the decision boundary to the closest training points; if you also report the full width between supporting boundaries, label it separately.

$w_1 x_1 + w_2 x_2 + b = 0 \Rightarrow$ find w_1, w_2, b

1: $(1,1)$ & $y=+1 \Rightarrow w_1 + w_2 + b = +1$ (1)

5: $(1,0)$ & $y=-1 \Rightarrow w_1 + b = -1$ (2)

6: $(0,1)$ & $y=-1 \Rightarrow w_2 + b = -1$ (3)

$\therefore w_2 = w_1$ due to equations 2 & 3 and also:

$(w_1 + w_2 + b) - (w_1 + b) = 1 - (-1)$

$\therefore w_2 = 2, \quad w_1 = 2$ and therefore:

$w_1 + b = -1 \Rightarrow 2 + b = -1 \Rightarrow \underline{b = -3}$

So, $w = (2,2)$, $b = -3$, boundary is $2x_1 + 2x_2 - 3 = 0$

(check points are on correct side: $(x_1 + x_2 = 1.5)$)

point	x	y	$w \cdot x + b$	$y(w \cdot x + b)$
1	(1,1)	+1	1	1 (support vector!)
2	(1,2)	+1	3	3
3	(2,1)	+1	3	3
4	(0,0)	−1	−3	3
5	(1,0)	−1	−1	1 (support vector)
6	(0,1)	−1	−1	1 (support vector)

1 for a supp. vec ↑

Geometric margin: Distance from point to line $wx + b = 0$: $\frac{|wx+b|}{\|w\|}$

$\therefore \|w\| = \sqrt{2^2 + 2^2} = 2\sqrt{2}, \quad \therefore \frac{1}{\|w\|} = \frac{1}{2\sqrt{2}} = \frac{\sqrt{2}}{4} \approx \boxed{0.354}$

↳ boundary to closest point distance would be $2/\|w\| = \sqrt{2}/2 \approx \boxed{0.707}$

Question 2: K-means — 30 points

(a) Implement the update steps — 15 points.

In kmeans_template.py, complete update_assignments() and update_centroids() from scratch. NumPy operations are allowed; do not substitute a library K-means implementation. The outer loop is provided. Document how you resolve distance ties and handle empty clusters.

For resolving the ties, basically if you come across a point that's the same distance away from two or more centroids, the np.argmin just returns the first minimum, which is the lowest indexed centroid. So the same input always gives the same answer. In terms of empty clusters, if no points are assigned to a centroid then it stays where it was. Without this consideration you'd just take the average of zero points which is not ideal. The empty clusters occur when centroids are selected with replacement so two can be the same point, so then the tie handler gives all their points to the lower index, and the other one starts empty.

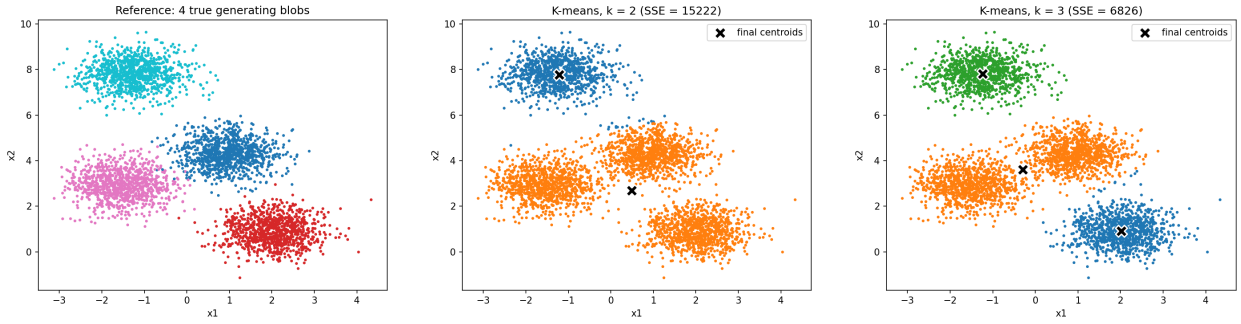
(b)Run and plot — 15 points.

Generate the provided toy dataset. Run your implementation for k = 2, 3, 4, 6, 10. Save a plot of the cluster assignments and final centroids for each value. In a short paragraph, explain what changes as k increases and how the results relate to the four generating groups.

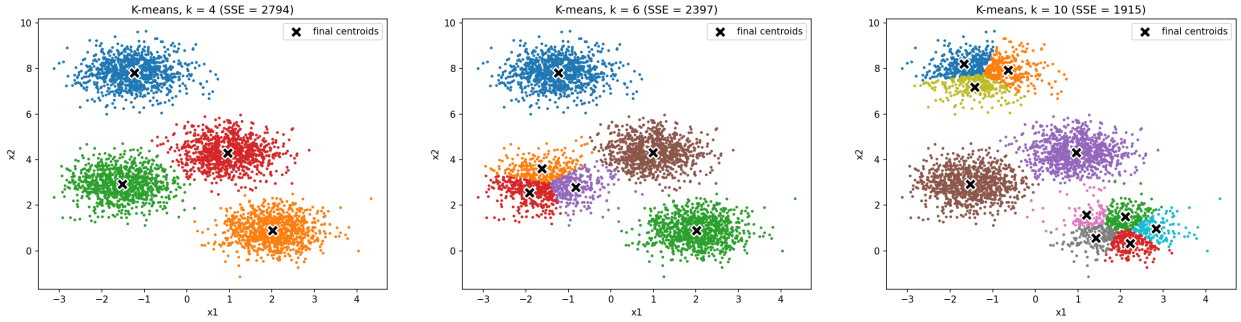
k	SSE	Decrease from previous k	Cluster sizes
2	15222.2	—	1017, 2983
3	6826.1	8396.1	1008, 1992, 1000
4	2794.2	4031.9	1000, 998, 1002, 1000
6	2396.8	397.4	1000, 299, 997, 407, 319, 978
10	1914.8	482.0	365, 350, 253, 261, 988, 997, 123, 213, 285, 165

Table 1: K-means results for each k (seed 0). Each true blob contains 1000 points.

Discussion. When k is smaller than 4, K-means has too few centroids so it merges true blobs that are close together. Then a centroid ends up in the empty space between them, then at k = 4 there is one centroid per blob, each centroid lands in the middle of a blob. So when k is larger than 4, K-means has excess centroids so it just splits real blobs into smaller pieces, and which blobs get split depends on the random starting points. SSE keeps going down as k grows, because more centroids always means points are closer to some centroid, but this result starts fast and then tapers off and "converges" which means you've reached the natural number of clusters that matches how the data was generated.



(a) True blobs (reference) (b) $k = 2$ (c) $k = 3$



(d) $k = 4$ (e) $k = 6$ (f) $k = 10$

Figure 1: Cluster assignments (colors) and final centroids (black $\times$) for each k, with the true generating blobs shown for reference.

Question 3: Noisy-label SVM and K-NN repair — 40 points

(a) Noisy-label baseline — 20 points.

Complete corrupt_training_labels() and the baseline portion of run_comparison(). Independently flip each original positive training label with probability0.35, and each original negative training label with probability 0.20. Apply the probabilities to the original labels so that an example cannot be flipped twice.

Train an RBF SVM with C =1 and gamma = 0.01. Keep the generated training and test sets fixed, repeat label corruption and training 20 times, and report the mean accuracy on the clean test set. Use a seed for reproducibility.

The mean clean-test accuracy of the noisy-label baseline over 20 trials was 0.8280 (range 0.7840–0.8470).

(b) K-NN label repair — 20 points.

Complete repair_labels_knn() and the comparison portion of run_comparison()

For each training point: (1) Find its five nearest other training points using Euclidean distance. (2) Replace its label with the majority of those neighbors' noisy labels. (3) Compute every repaired label from the same original noisy-label array, rather than modifying labels sequentially.

A neighbor-search library is allowed. Five binary votes cannot tie; use a consistent rule if distances tie when selecting neighbors. Train a fresh SVM with the same parameters on the repaired labels. For each trial, use the same features and corrupted labels as the baseline.

Save a CSV with one row per trial and columns trial, baseline_accuracy, and knn_accuracy. Report both mean accuracies and their difference. Briefly explain whether K-NN repair helped and why it might help or hurt. Improvement is not guaranteed. No parameter sweep or additional clustering-based repair experiment is required.

The repair method may use only training features and noisy training labels. Do not pass it clean training labels or test labels. K-NN is a preprocessing step here; the final classifier is an SVM.

K-NN repair raised the mean clean-test accuracy from 0.8280 **to 0.8644 which is a gain of +0.036, and it beat the baseline in all 20 trials. It helped because labels are flipped at random places, while the true classes are grouped together. A flipped point far into its own class has mostly correct neighbors so the vote fixes it. These wrong labels are also the ones that pull the SVM boundary the most. And repair can also hurt because when the two classes overlap, the neighbors are mixed so the vote can change correct labels into wrong ones. Positives are flipped more often, so the vote can turn true positives near the boundary into negatives. So if more than half the labels in an area were wrong, repair would make things worse**